

CSW306 — BACKEND DEVELOPMENT

Mẫu điền sẵn mang vào phòng thi

Quy trình đầy đủ SQL Server ⇌ Visual Studio ⇌ Swagger, kèm template code chứa chỗ điền và bảng quyết định dùng nhánh nào theo từng đề.

Quy ước: chỗ nào viết `___TÊN_IN_HOA___` nghĩa là phải thay bằng giá trị thật lấy từ đề thi trước khi copy vào Visual Studio. Đọc kỹ Phần B (bảng quyết định) trước khi điền.

A. Quy trình đầy đủ: SSMS ⇌ Visual Studio ⇌ Swagger

Làm tuần tự từ trên xuống. Nhấn màu cho biết đang thao tác ở công cụ nào.

A1. Khởi tạo project (Visual Studio)

- [VS] 1. Mở Visual Studio → Create a new project → chọn "ASP.NET Core Web API" → Next.
- [VS] 2. Project name = `___DDMMYYYY_BE_HoTen___` (đúng convention đề yêu cầu) → chọn nơi lưu → Next.
- [VS] 3. Framework chọn .NET 8.0 (hoặc bản đề yêu cầu) → Authentication type: None → giữ nguyên "Use controllers" đã tick sẵn → Create.
- [VS] 4. Tools → NuGet Package Manager → Package Manager Console, chạy lần lượt từng dòng:

Package Manager Console

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.AspNetCore.Authentication.JwtBearer
# Swashbuckle.AspNetCore (Swagger) thường có sẵn trong template, không có thì cài thêm
```

- [VS] 5. Trong Solution Explorer, chuột phải vào tên project → Add → New Folder, tạo lần lượt: Models, Data, Repositories, Controllers, DTOs.

A2. Xác định SQL Server đang chạy (SSMS)

- [SSMS] 6. Mở SQL Server Management Studio → màn hình Connect to Server.
- [SSMS] 7. Server name: thử `___.` (dấu chấm) hoặc `___(localdb)\\MSSQLLocalDB___` hoặc `___.\SQLEXPRESS___` tùy máy đã cài bản nào → Connect.
→ Tên instance hiện ở góc trên Object Explorer sau khi connect thành công — copy đúng chuỗi này để dán vào connection string ở bước A4.
- [SSMS] 8. KHÔNG cần tạo Database thủ công bằng tay — Migration ở bước A6 sẽ tự tạo. Chỉ cần nhớ chọn 1 cái TÊN database sẽ dùng.

A3. Viết Model & ApplicationDbContext (Visual Studio)

- [VS] 9. Điền code Model theo Phần C1 vào Models/ — đối chiếu đúng bằng Field/Type/Constraints trong đề.
- [VS] 10. Điền ApplicationDbContext theo Phần C2 vào Data/.

A4. Cấu hình kết nối Database (Visual Studio)

- [VS] 11. Mở appsettings.json → điền connection string (Phần C2) với Server lấy từ bước A2, Database = tên bạn tự đặt.
- [VS] 12. Mở Program.cs → điền đoạn AddDbContext + AddAuthentication JWT (Phần C2).

A5. Chạy Migration & kiểm tra ngược lại trong SSMS

- [VS] 13. Package Manager Console, chạy:

```
Add-Migration InitialCreate
Update-Database
```

- [SSMS] 14. Chuột phải vào "Databases" trong Object Explorer → Refresh.

[SSMS] 15. Mở rộng database vừa tạo → Tables → đếm đủ số bảng đúng số entity trong đề chưa.

⚠ Không thấy bảng nào? Quay lại VS, xem tab "Error List" hoặc output của Update-Database — thường do sai connection string ở A4. Sửa xong chạy lại Update-Database, KHÔNG cần Add-Migration lại.

A6. Repository & Controller (Visual Studio)

[VS] 16. Điền Repository theo Phần C3 (đọc Phần B để biết dùng bản cụ thể hay generic) vào Repositories/, đăng ký DI trong Program.cs.

[VS] 17. Điền Controller CRUD + CategoriesController theo Phần C4 vào Controllers/.

[VS] 18. Điền AuthController theo Phần C5 vào Controllers/.

[VS] 19. Điền OrdersController theo Phần C6 — đây là phần khó nhất, đọc kỹ Phần B trước khi điền.

A7. Tạo user test (SSMS) — trước khi chạy thử

[SSMS] 20. Trong Object Explorer, chuột phải đúng database vừa tạo → New Query.

[SSMS] 21. Dán câu lệnh INSERT (Phần C5) vào cửa sổ Query → bấm Execute (hoặc F5).

[SSMS] 22. Gõ SELECT * FROM Users; → Execute → kiểm tra 2 dòng user vừa tạo hiện ra đúng.

A8. Chạy thử & test trên Swagger

[VS] 23. Bấm nút Run màu xanh (hoặc F5) ở thanh công cụ trên cùng → trình duyệt tự mở Swagger UI (<https://localhost:xxxx/swagger>).

[SWAGGER] 24. Mở POST /api/auth/login → Try it out → dán JSON {"email":"admin@test.com","password":"..."} → Execute → copy giá trị "token" trong Response body.

[SWAGGER] 25. Bấm nút ổ khóa "Authorize" ở góc trên phải trang Swagger → gõ đúng cú pháp: Bearer ____dán_token_vào_đây____ → Authorize → Close.

[SWAGGER] 26. Test lần lượt từng endpoint GET/POST/PUT/DELETE, đối chiếu route trong đề. Endpoint public thử KHÔNG bấm Authorize trước để chắc chắn không cần token vẫn chạy.

[SSMS] 27. Sau khi POST/PUT/DELETE thử nghiệm, quay lại SSMS chạy SELECT để xác nhận dữ liệu đã lưu đúng vào bảng.

A9. Backup & nộp bài

[SSMS] 28. Chuột phải vào database → Tasks → Back Up... → Backup type: Full → Add đường dẫn lưu file .bak → OK.

[VS] 29. Đóng Visual Studio, zip toàn bộ folder project (có thể xóa bin/ và obj/ cho nhẹ) + file .bak vừa backup.

[VS] 30. Đặt tên file zip đúng convention đề yêu cầu → nộp bài.

B. Bảng quyết định — đề ghi gì thì dùng nhánh nào

Đọc kỹ đề trước khi điền code ở Phần C. Đề Q1/Q3/Lab7/Lab8 không hoàn toàn giống nhau ở những điểm sau — chọn đúng nhánh để không bị lệch yêu cầu.

Repository — cụ thể (specific) hay generic?

Dấu hiệu trong đề	Dùng cách nào
Đề liệt kê sẵn chữ ký: Task<List<X>> GetAll(); Task<X?> GetById(int id); ... (như Q1/Q3)	Dùng bản Repository CỤ THỂ (Phần C3 — bản A). Copy y chang tên method, KHÔNG thêm "Async" nếu đề không viết.
Đề ghi rõ: "Use the IRepository<T> interface" (như Lab 8)	Dùng bản GENERIC IRepository<T> (Phần C3 — bản B).

Role/quyền — tên chuỗi nào?

Dấu hiệu trong đề	Dùng cách nào
Bảng User trong đề ghi: Role NVARCHAR(20) NOT NULL: 'Admin' / 'User'	Dùng chuỗi "Admin" và "User" trong [Authorize(Roles="...")] — đúng chữ hoa/thường.
Bảng User ghi: Role ('Admin', 'Customer')	Dùng chuỗi "Admin" và "Customer" thay vì "User".

Response GET danh sách — nested hay flat?

Dấu hiệu trong đề	Dùng cách nào
Đề ghi rõ chỉ 1 định dạng (VD chỉ nested {category: {...}})	Bám đúng định dạng đề yêu cầu, không tự đổi.
Đề ghi "(both formats accepted)" như Q1	Ưu tiên dùng FLAT (categoryId + categoryName) — code ngắn hơn, ít lỗi hơn nested.

GET orders/user/{userId} — Admin có được xem hộ không?

Dấu hiệu trong đề	Dùng cách nào
Đề ghi: "userId in route must match userId in JWT token" — không nhắc ngoại lệ nào	KHÔNG cho Admin bypass ở route này. Admin xem hết đơn qua route riêng GET /api/orders (Admin-only).
Đề ghi rõ Admin được xem MỌI order kể cả qua route theo userId	Mới thêm nhánh kiểm tra Role == "Admin" để bypass — đọc đúng câu chữ, đừng tự suy diễn thêm quyền không được yêu cầu.

Password — có bắt buộc hash không?

Dấu hiệu trong đề	Dùng cách nào
Đề ghi: "Passwords can be stored as plain text for testing (no hashing required)"	Lưu plain text, KHÔNG cần cài BCrypt — tiết kiệm thời gian, đúng yêu cầu.
Đề ghi: "Hash user passwords using BCrypt or ASP.NET Identity" là REQUIREMENT bắt buộc	Cài package BCrypt.Net-Next, hash password trước khi lưu và khi so sánh lúc login.

Đặt tên project khi tạo

Dấu hiệu trong đề	Dùng cách nào
Đề ghi format DateTime(DD-MM-YYYY)_BE_YourName	Đặt tên có DẤU GẠCH NGANG giữa ngày-tháng-năm đúng như mẫu, ví dụ 19-12-2025_BE_NguyenXuanCuong.
Đề ghi format DDMMYYYY_BE_YourName (không dấu gạch)	Đặt liền không dấu gạch, ví dụ 19122025_BE_NguyenXuanCuong.

C. Template code — chờ chỗ điền theo đề

C1. Model

```
Models/___TenEntity___.cs (VD: Fruit.cs / Pet.cs / Book.cs)
public class ___TenEntity___
{
    public int Id { get; set; }

    [Required, MaxLength(___SoKyTuToiDa___)]
    public string ___TenField1___ { get; set; }    // VD: Name

    [Required]
    [Range(0.01, double.MaxValue, ErrorMessage = "___ThongBaoLoi___")]
    public decimal ___TenFieldGia___ { get; set; }    // VD: Price

    // Them field khác theo dung bang trong de tai day, VD:
    // public ___KieuDuLieu___ ___TenField___ { get; set; }

    public int Quantity { get; set; } = 0;    // xoa dong nay neu de khong yeu cau

    [Required]
    public int ___TenBangChaId___ { get; set; }    // VD: CategoryId
    [ForeignKey("___TenBangChaId___")]
    public ___TenLopCha___ ___TenBangCha___ { get; set; }    // VD: Category Category

    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

public class ___TenLopCha___    // VD: Category
{
    public int Id { get; set; }

    [Required, MaxLength(___SoKyTu___)]
    public string Name { get; set; }

    [MaxLength(255)]
    public string? Description { get; set; }

    public ICollection<___TenEntity___> ___TenEntity___s { get; set; } = new List<___TenEntity___>();
}

public class User
{
    public int Id { get; set; }

    [Required, MaxLength(200)]
    public string FullName { get; set; }

    [Required, EmailAddress, MaxLength(255)]
    public string Email { get; set; }

    [Required]
    public string Password { get; set; }

    [Required]
    public string Role { get; set; }    // "Admin"/"User" hoac "Admin"/"Customer" - xem Phan B

    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

public class Order
{
    public int Id { get; set; }

    [Required]
    public int UserId { get; set; }
    [ForeignKey("UserId")]
    public User User { get; set; }

    public DateTime OrderDate { get; set; }
    public decimal TotalAmount { get; set; }
    public string Status { get; set; } = "Pending";
    public DateTime CreatedAt { get; set; } = DateTime.Now;
}
```

```

    public ICollection<OrderItem> OrderItems { get; set; } = new List<OrderItem>();
}

public class OrderItem
{
    public int Id { get; set; }

    [Required]
    public int OrderId { get; set; }
    [ForeignKey("OrderId")]
    public Order Order { get; set; }

    [Required]
    public int ____TenEntity____Id { get; set; } // VD: FruitId / PetId
    [ForeignKey("____TenEntity____Id")]
    public ____TenEntity____ ____TenEntity____ { get; set; }

    [Range(1, int.MaxValue)]
    public int Quantity { get; set; }

    public decimal UnitPrice { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

```

C2. AppDbContext + appsettings.json + Program.cs

Data/AppDbContext.cs

```

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<____TenEntity____> ____TenEntity____s { get; set; }
    public DbSet<____TenLopCha____> ____TenLopCha____s { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<____TenLopCha____>().HasIndex(c => c.Name).IsUnique();
        modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique();
        base.OnModelCreating(modelBuilder);
    }
}

```

appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=____LayTuBuocA2____;Database=____TenDatabase____;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Jwt": {
    "Key": "____it_nhat_32_ky_tu_bat_ky____",
    "Issuer": "ExamApi",
    "Audience": "ExamApiUsers",
    "ExpiryMinutes": 120
  }
}

```

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// ____DangKyRepositoryODay____ (xem Phan C3)
// builder.Services.AddScoped<I____TenEntity____Repository, ____TenEntity____Repository>();

var jwtSettings = builder.Configuration.GetSection("Jwt");
var key = Encoding.UTF8.GetBytes(jwtSettings["Key"]);

```

```
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(key)
    };
});
builder.Services.AddAuthorization();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthentication(); // LUON truoc Authorization
app.UseAuthorization();
app.MapControllers();
app.Run();
```

C3-A. Repository — bản CỤ THỂ (dùng khi đề cho sẵn chữ ký method)

```

Repositories/I__TenEntity__Repository.cs + __TenEntity__Repository.cs
public interface I__TenEntity__Repository
{
    __SaoChepChuKyMethodTuDe__
    // VD: Task<List<Fruit>> GetAll();
    //      Task<Fruit?> GetById(int id);
    //      Task Add(Fruit fruit);
    //      Task Update(Fruit fruit);
    //      Task Delete(int id);
    //      Task<List<Fruit>> GetByCategoryId(int categoryId);
}

public class __TenEntity__Repository : I__TenEntity__Repository
{
    private readonly AppDbContext _context;
    public __TenEntity__Repository(AppDbContext context) => _context = context;

    public async Task<List<__TenEntity__>> GetAll() =>
        await _context.__TenEntity__s.Include(x => x.__TenBangCha__).ToListAsync();

    public async Task<__TenEntity__?> GetById(int id) =>
        await _context.__TenEntity__s.Include(x => x.__TenBangCha__)
            .FirstOrDefaultAsync(x => x.Id == id);

    public async Task Add(__TenEntity__ entity)
    {
        _context.__TenEntity__s.Add(entity);
        await _context.SaveChangesAsync();
    }

    public async Task Update(__TenEntity__ entity)
    {
        _context.__TenEntity__s.Update(entity);
        await _context.SaveChangesAsync();
    }

    public async Task Delete(int id)
    {
        var entity = await _context.__TenEntity__s.FindAsync(id);
        if (entity != null)
        {
            _context.__TenEntity__s.Remove(entity);
            await _context.SaveChangesAsync();
        }
    }

    public async Task<List<__TenEntity__>> GetByCategoryId(int categoryId) =>
        await _context.__TenEntity__s.Where(x => x.__TenBangChaId__ == categoryId).ToListAsync();
}

```

C3-B. Repository — bản GENERIC (dùng khi đề yêu cầu IRepository<T>)

```

public interface IRepository<T> where T : class
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T?> GetByIdAsync(int id);
    Task<T> AddAsync(T entity);
    Task<T?> UpdateAsync(int id, T entity);
    Task<bool> DeleteAsync(int id);
}

public class __TenEntity__Repository : IRepository<__TenEntity__>
{
    private readonly AppDbContext _context;
    public __TenEntity__Repository(AppDbContext context) => _context = context;

    public async Task<IEnumerable<__TenEntity__>> GetAllAsync() =>
        await _context.__TenEntity__s.ToListAsync();

    public async Task<__TenEntity__?> GetByIdAsync(int id) =>
        await _context.__TenEntity__s.FindAsync(id);

    public async Task<__TenEntity__> AddAsync(__TenEntity__ entity)
    {
        _context.__TenEntity__s.Add(entity);
    }
}

```

```

        await _context.SaveChangesAsync();
        return entity;
    }

    public async Task<__TenEntity__?> UpdateAsync(int id, __TenEntity__ entity)
    {
        var existing = await _context.__TenEntity__s.FindAsync(id);
        if (existing == null) return null;
        // __GanTungField0Day__ (không dùng _context.Update() nguyên khối)
        await _context.SaveChangesAsync();
        return existing;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var existing = await _context.__TenEntity__s.FindAsync(id);
        if (existing == null) return false;
        _context.__TenEntity__s.Remove(existing);
        await _context.SaveChangesAsync();
        return true;
    }
}

// Đăng ký DI trong Program.cs:
// builder.Services.AddScoped<IRepository<__TenEntity__>, __TenEntity__Repository>();

```

C4. Controller CRUD chính + Categories

Controllers/___TenEntity___sController.cs

```
[ApiController]
[Route("api/[controller]")]
public class ___TenEntity___sController : ControllerBase
{
    private readonly I___TenEntity___Repository _repo; // hoặc IRepository<___TenEntity___> _repo;
    private readonly AppDbContext _context;
    public ___TenEntity___sController(I___TenEntity___Repository repo, AppDbContext context)
    {
        _repo = repo; _context = context;
    }

    [HttpGet]
    [AllowAnonymous]
    public async Task<IActionResult> GetAll()
    {
        var list = await _repo.GetAll();
        var result = list.Select(x => new {
            x.Id, x.___TenField1___, x.___TenFieldGia___,
            ___TenBangChaId___ = x.___TenBangChaId___,
            ___TenBangCha___Name = x.___TenBangCha___?.Name
        });
        return Ok(result);
    }

    [HttpGet("{id}")]
    [AllowAnonymous]
    public async Task<IActionResult> GetById(int id)
    {
        var entity = await _repo.GetById(id);
        if (entity == null) return NotFound(new { message = $"___TenEntity___ {id} not found" });
        return Ok(entity);
    }

    [HttpPost]
    [Authorize(Roles = "___VaiTroAdmin___")] // xem Phan B: "Admin"
    public async Task<IActionResult> Create([FromBody] ___TenEntity___ entity)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        if (!await _context.___TenLopCha___s.AnyAsync(c => c.Id == entity.___TenBangChaId___))
            return BadRequest(new { message = "___TenBangChaId___ không tồn tại" });

        await _repo.Add(entity);
        return CreatedAtAction(nameof(GetById), new { id = entity.Id }, entity);
    }

    [HttpPut("{id}")]
    [Authorize(Roles = "___VaiTroAdmin___")]
    public async Task<IActionResult> Update(int id, [FromBody] ___TenEntity___ entity)
    {
        if (id != entity.Id) return BadRequest(new { message = "Id không khớp" });
        if (!ModelState.IsValid) return BadRequest(ModelState);

        var existing = await _repo.GetById(id);
        if (existing == null) return NotFound();

        await _repo.Update(entity);
        return NoContent();
    }

    [HttpDelete("{id}")]
    [Authorize(Roles = "___VaiTroAdmin___")]
    public async Task<IActionResult> Delete(int id)
    {
        var existing = await _repo.GetById(id);
        if (existing == null) return NotFound();

        await _repo.Delete(id);
        return NoContent();
    }
}
```

Controllers/___TenLopCha___sController.cs

```
[ApiController]
[Route("api/[controller]")]
```

```
public class ___TenLopCha___sController : ControllerBase
{
    private readonly AppDbContext _context;
    public ___TenLopCha___sController(AppDbContext context) => _context = context;

    [HttpGet]
    [AllowAnonymous]
    public async Task<IActionResult> GetAll() =>
        Ok(await _context.___TenLopCha___s.ToListAsync());
}
```

C5. AuthController + tạo user test

Controllers/AuthController.cs

```
[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly IConfiguration _config;
    public AuthController(AppDbContext context, IConfiguration config)
    {
        _context = context; _config = config;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginDto dto)
    {
        var user = await _context.Users.FirstOrDefaultAsync(u => u.Email == dto.Email);
        if (user == null || user.Password != dto.Password) // __doi_thanh_BCrypt.Verify_neu_de_bat_hash__
            return Unauthorized(new { message = "Sai email hoặc password" });

        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new Claim(ClaimTypes.Email, user.Email),
            new Claim(ClaimTypes.Role, user.Role)
        };

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.Now.AddMinutes(double.Parse(_config["Jwt:ExpiryMinutes"])),
            signingCredentials: creds
        );

        return Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token), role = user.Role });
    }
}

public class LoginDto { public string Email { get; set; } public string Password { get; set; } }
```

SQL – chạy trong SSMS (New Query trên đúng database)

```
INSERT INTO Users (FullName, Email, Password, Role, CreatedAt)
VALUES
('Admin Test', 'admin@test.com', 'admin123', '__VaiTroAdmin__', GETDATE()),
('User Test', 'user@test.com', 'user123', '__VaiTroUser__', GETDATE());
-- __VaiTroAdmin__ / __VaiTroUser__ = "Admin"/"User" hoặc "Admin"/"Customer", xem Phan B

SELECT * FROM Users;
```

C6. OrdersController — phần khó nhất

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly AppDbContext _context;
    public OrdersController(AppDbContext context) => _context = context;

    [HttpGet]
    [Authorize(Roles = "___VaiTroAdmin___")]
    public async Task<IActionResult> GetAllOrders()
    {
        var orders = await _context.Orders.Include(o => o.OrderItems).ToListAsync();
        return Ok(orders);
    }

    [HttpPost]
    [Authorize]
    public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDto dto)
    {
        var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
        if (userIdClaim == null) return Unauthorized();
        int userId = int.Parse(userIdClaim);

        var order = new Order { UserId = userId, OrderDate = DateTime.Now, Status = "Pending" };

        decimal total = 0;
        foreach (var i in dto.Items)
        {
            var entity = await _context.___TenEntity___s.FindAsync(i.___TenEntity___Id);
            if (entity == null)
                return BadRequest(new { message = $"___TenEntity___ {i.___TenEntity___Id} không tồn tại" });
            if (i.Quantity <= 0)
                return BadRequest(new { message = "Quantity phải > 0" });
            if (entity.Quantity < i.Quantity)
                return BadRequest(new { message = $"Không đủ hàng cho {entity.___TenField1___}" });

            var orderItem = new OrderItem
            {
                ___TenEntity___Id = entity.Id,
                Quantity = i.Quantity,
                UnitPrice = entity.___TenFieldGia___ // gia HIEN TAI, không phải gia đã lưu sẵn
            };
            total += orderItem.Quantity * orderItem.UnitPrice;
            order.OrderItems.Add(orderItem);
            entity.Quantity -= i.Quantity;
        }
        order.TotalAmount = total;

        _context.Orders.Add(order);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(GetOrderById), new { id = order.Id }, order);
    }

    [HttpGet("{id}")]
    [Authorize]
    public async Task<IActionResult> GetOrderById(int id)
    {
        var order = await _context.Orders.Include(o => o.OrderItems).FirstOrDefaultAsync(o => o.Id == id);
        if (order == null) return NotFound();
        return Ok(order);
    }

    [HttpGet("user/{userId}")]
    [Authorize]
    public async Task<IActionResult> GetByUser(int userId)
    {
        var tokenUserId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
        if (tokenUserId != userId.ToString()) return Forbid();
        // ___NEU_DE_CHO_PHEP_ADMIN_XEM_HO___: thêm điều kiện Role=="___VaiTroAdmin___" ở đây (xem Phan B)

        var orders = await _context.Orders.Include(o => o.OrderItems)
            .Where(o => o.UserId == userId).ToListAsync();
        return Ok(orders);
    }
}

public class CreateOrderDto { public List<OrderItemDto> Items { get; set; } }
```

```
public class OrderItemDto { public int __TenEntity__Id { get; set; } public int Quantity { get; set; } }
```

✓ **5 bước bắt buộc trong CreateOrder:** 1) UserId từ JWT, không từ body. 2) OrderDate = DateTime.Now. 3) Status = "Pending". 4) UnitPrice = giá HIỆN TẠI của entity. 5) TotalAmount = tổng (Quantity × UnitPrice).